

边云协同下的计算卸载与资源分配策略*

张文柱, 石亚坤, 高杜梅

(西安建筑科技大学信息与控制工程学院, 陕西 西安 710055)

摘要: 在车联网中, 车辆有限的计算能力、边缘服务器动态的计算资源和云服务器遥远的部署位置使得合理设计计算任务卸载与资源分配方案极具挑战。针对上述挑战, 以最小化处理计算任务的时延与能耗的加权和为目标, 提出了一种基于深度强化学习的联合计算卸载与资源分配的算法。具体来说, 为了实现边云协作处理计算任务, 首先设计了基于软件定义网络边云协同的网络架构, 并给出了任务优先级的度量标准; 其次分别建立了云-边-端任务的计算模型; 然后设计了优化系统延迟与能耗的目标函数, 并将其转化为系统效用函数; 最后利用深度强化学习算法依据系统效用来决定计算任务的卸载与资源分配策略。实验结果表明, 与现有的算法相比, 所提算法在降低系统时延与能耗以及提高任务计算成功率方面的表现显著优于基准算法。

关键词: 计算卸载; 资源分配; 云计算; 移动边缘计算; 深度强化学习

中图分类号: TN929.5

文献标志码: A

doi: 10.3969/j.issn.1007-130X.2026.03.003

A computing offloading and resource allocation strategy under edge-cloud collaboration

ZHANG Wenzhu, SHI Yakun, GAO Dumei

(College of Information and Control Engineering, Xi'an University of Architecture and Technology, Xi'an 710055, China)

Abstract: In the Internet of Vehicles (IoV), the limited computational capabilities of vehicles, the dynamic computational resources of edge servers, and the remote deployment locations of cloud servers pose significant challenges in designing computation offloading and resource allocation schemes. This paper proposes a deep reinforcement learning-based joint computation offloading and resource allocation algorithm, aiming at minimizing the weighted sum of latency and energy consumption for processing computational tasks. Specifically, to enable collaborative processing of computational tasks between edge and cloud servers, a software-defined networking (SDN) based edge-cloud collaborative network architecture is first designed, along with a metric for task priority. Subsequently, computational models for cloud-edge-end device tasks are established separately. Then, an objective function is designed to optimize system latency and energy consumption, which is transformed into a system utility function. Finally, a deep reinforcement learning algorithm is utilized to determine computation offloading and resource allocation strategies based on the system utility. Experimental results demonstrate that, compared to existing algorithms, the proposed algorithm significantly outperforms benchmark algorithms in reducing system latency and energy consumption, as well as improving the success rate of task computation.

Key words: computation offloading; resource allocation; cloud computing; mobile edge computing;

* 收稿日期: 2024-04-18; 修回日期: 2024-10-11

基金项目: 陕西省重点研发计划(2025CY-YBXM-063)

通信作者: 石亚坤(shiyakun@xauat.edu.cn)

引用格式: 张文柱, 石亚坤, 高杜梅. 边云协同下的计算卸载与资源分配策略[J]. 计算机工程与科学, 2026, 48(3): 398-410.

Citation: ZHANG Wenzhu, SHI Yakun, GAO Dumei. A computing offloading and resource allocation strategy under edge-cloud collaboration[J]. Computer Engineering & Science, 2026, 48(3): 398-410.

deep reinforcement learning

1 引言

在人工智能和物联网两大技术经历蓬勃发展和长足进步的同时,车联网 IoV(Internet of Vehicles)已经由简单的交互系统向智能交通系统 ITS(Intelligent Transportation System)转变。随着 ITS 的快速发展,交通环境变得更加复杂,交通环境中的车辆与车辆、车辆与固定接入点以及车辆与行人之间的通信形成了一个开放的车载自组织网络 VANET(Vehicular Ad hoc NETwork)。ITS 依靠车载传感器和雷达装置,实时获取车辆自身状况及其周边环境的数据信息,并且将采集到的信息共享给其他车辆或者对其进行加工处理,应用于智能交通系统的控制决策,从而为更安全的道路驾驶做出贡献。车辆与路边基础设施之间的互联 V2I(Vehicle to Infrastructure)作为实现 ITS 的基础,而复杂交通环境形成的 VANET 恰好提供了这一基础。VANET 作为 ITS 的关键技术手段之一,可实现自动驾驶、车路协同和人车动态交互等智能应用服务。在复杂交通网络环境中,上述多种智能应用服务在不同对象间的互联会产生许多持续、高速和具有实时性的数据流^[1],计算复杂度高而且数据密集。各种智能服务应用虽然极大地提升了车辆用户的体验,但也给资源受限的车辆带来了严峻挑战,尤其是在支持那些计算需求高且对时延敏感的应用时更为明显。因此,如何解决车辆计算能力不足的问题,已经成为目前研究的焦点。

要使车辆能够支持计算密集型和延迟敏感型的计算任务,需要大量的计算资源,而车辆可用的计算资源往往不足以完成这样的计算任务。在这种情况下,必须使用计算卸载技术将任务转移到其他设备上执行。通过云计算 CC(Cloud Computing)方式将任务卸载到云服务器进行计算,是减轻车辆计算负担的有效方法^[2]。CC 虽然可以应对车辆计算能力不足的挑战,但传输大量数据会导致高时延和连接不稳定的问题。为减轻核心网络的带宽压力,降低车载计算任务的执行时延并实现对庞大数量的车辆与基础设施的良好管理,将移动边缘计算 MEC(Mobile Edge Computing)与软件定义网络 SDN(Software Defined Network)技术引入车联网是很好的解决办法。MEC 的引入可以解决 CC 传输时延高的问题,SDN 技术的引入则能够提供一种强有力的信息采集和资源管理工具,是提升

车辆边缘计算网络效率很好的补充^[3]。

作为一种新兴的计算模式,MEC 中计算和存储资源被部署到更接近移动设备的地方,可以为车辆提供辅助计算的服务。车辆与边缘服务器之间距离较短,所以 MEC 可以带来低时延的优势。SDN 的主要思想是将网络的控制平面与数据平面分离^[4]。这种分离提供的优势主要包括:1)使得遵循 OpenFlow 协议的硬件设备仅需要负责数据转发的任务,而负责网络操作系统功能的任务则是交由控制层完成,这样可以实现网络流量的灵活控制;2)通过抽象底层网络基础设施,SDN 可以拥有网络全局视野,实现集中控制并进行统一资源管理,在 MEC 中,借助 SDN 控制器对网络设备进行集中管理和控制,可以为网络连接和服务管理提供简便的解决办法;3)网络中存在较多移动车辆 MV(Moving Vehicle)和边缘计算设备,要想在逻辑上控制车载计算任务的卸载过程,需要借助 SDN 控制器对多网络设备进行控制,并且在控制器上运行计算卸载与资源分配决策方法。

设计有效的计算卸载决策方案是弥补车辆计算能力不足的关键,故需要考虑以下 3 个问题:1)任务通过什么方式进行卸载? 2)任务卸载到哪个计算节点? 3)如何给卸载的任务分配通信和计算资源? 由于要同时考虑卸载决策变量、通信资源分配决策变量和计算资源分配决策变量,所以计算卸载问题本质上是涉及多个决策变量的优化问题,该问题能够被转化为混合整数非线性规划问题。使用传统的启发式算法直接求解该问题面临以下挑战:1)这些算法常常需要大量先验知识;2)即便有了先验知识,它们也需要经过多次迭代才能得到令人满意的局部最优解;3)这些算法在处理试错与探索性问题时能力有限;4)这些算法难以直接应用到需要对环境变化做出实时响应的场景。如何有效地为计算节点分配任务和高效利用边缘服务器计算资源,这对于降低任务处理时延与能耗,以及提高任务计算成功率来说都是至关重要的。基于深度强化学习 DRL(Deep Reinforcement Learning)算法可以适应复杂多变且需要实时做出决策的场景,具有自主学习、试错和探索的能力,能通过神经网络获得最优的决策。它是解决混合整数非线性规划问题的新方法,为解决 IoV 场景下的计算卸载与资源分配问题提供了新思路。

为了应对上述挑战,本文通过集成 SDN、MEC

以及 CC,设计了一种软件定义边缘云计算的三层车联网架构,用于边云协同下的计算卸载与资源分配决策。为了使系统效用最大化,提出 JCORA-DRL (Joint Computing Offloading and Resource Allocation based on Deep Reinforcement Learning) 算法,该算法可以处理连续动作空间,从而获得 IoV 环境下最优的计算卸载与资源分配策略。

本文的主要工作如下:

1)将 SDN 扩展到 IoV,在 SDN 辅助基于边云协同的车联网架构中,同时考虑二进制卸载和部分卸载,使得对于有不同计算需求的任务有更灵活的卸载决策。

2)考虑车辆流量随机性、不确定的通信条件以及边缘服务器时变计算资源,位于云端的控制器运行 JCORA-DRL 算法后,会依据计算任务的时延要求和优先级,将任务合理地调度到车辆本地、边缘服务器或云服务器上执行,最大化利用闲置计算资源,达到最高的计算成功率。

3)实验结果表明,与同类型的 DRL 算法以及与非强化学习算法相比,JCORA-DRL 算法在收敛速度和系统效用等方面具有更好的稳定性和性能。

本文其余部分组织如下,第 2 节中回顾并梳理了相关的工作;第 3 节中描述系统模型,并给出优化问题的形式化定义;第 4 节中详细介绍所提出的基于马尔可夫决策过程 MDP (Markov Decision Process) 的 JCORA-DRL 算法;第 5 节中给出所提算法的性能评估实验及数值结果;第 6 节总结全文。

2 相关工作

2.1 传统的启发式方法

文献[5]针对 MEC 中用户任务处理时延与能耗过高的问题,设计了云-边-端三层计算卸载架构,采用博弈论的方法得到了纳什均衡下的计算卸载与资源分配决策方案。文献[6]中将资源分配和计算卸载联合表述为非凸优化问题,利用博弈论和拉格朗日乘数方法,找到最优的资源分配与卸载决策。然而,这些方法需要大量迭代才能获得令人满意的局部最优解,这不适合环境快速变化且需要实时做出决策的应用场景。

2.2 基于神经网络的 DRL 算法

文献[7]考虑了云-边-端协同的网络架构,提出了一种基于多智能体的深度确定性策略梯度算法,通过最小化系统能耗,找到最佳的计算卸载与资源分配决策方案。文献[8]通过联合优化计算卸

载与资源分配决策,研究了一个端-边-云系统,在满足延迟约束的前提下,提出一种基于异步优势演员评论家 A3C (Asynchronous Advantage Actor-Critic) 算法,使任务卸载的效率和公平性最大化。DRL 优化方案可以通过强化学习神经网络获得接近最优的决策方案,但这些方案没有同时考虑时变信道状态和任务的随机性,以及任务的可分性与优先级。

2.3 基于 SDN 的方法

文献[9]构建了一种基于 SDN 的边云协作车联网计算卸载架构,把由时延和计算资源成本构成的效用作为优化目标,提出基于基于双深度 Q 网络 DDQN (Double Deep Q-Network) 的任务卸载机制和基于任务优先级的资源分配方案。文献[10]基于分布式计算与云-边-端一体化技术,提出一种高效的车载边缘计算卸载策略,来最小化系统的长期总成本,实现车辆边缘计算 VEC (Vehicle Edge Computing) 网络通感计算资源调度优化。IoV 中不同车辆的需求与网络状态变化频繁,SDN 能够快速响应这些变化,优化资源分配和卸载决策。此外,利用 SDN 的网络流量监控和控制功能,可以实现对计算任务的均衡负载。

与上述研究不同的是,本文在 IoV 场景下,考虑了任务可分性与优先级,兼顾二进制卸载与部分卸载,同时考虑了系统时变信道状态和边缘服务器的有限计算资源。通过 SDN 和虚拟化技术,将 IoV 系统划分为 3 个独立的层,引入 SDN 控制器作为协调器,发布控制指令以传输和处理信息,并提出基于 DRL 的联合计算卸载与资源分配方法。这弥补了传统启发式方法因需要大量迭代而无法在实时场景下快速决策的缺点。

3 系统模型与问题定义

表 1 本节符号

Table 1 Symbols in this section

符号	含义描述
MBS	微基站
RSU	路边单元
N	车辆数量
M	路边单元数量
$N_j(t)$	路边单元覆盖区域内车辆数量
s_i	计算负载
c_i	任务所需 CPU 周期

(续表)

符号	含义描述
t_d	任务容忍时延
$\alpha_i(t)$	二进制卸载决策变量
$\beta_i(t)$	部分卸载处理变量
$f_j^{\text{free}}(t)$	边缘服务器可用计算资源
p_i	车辆传输功率
B_j	路边单元带宽
$B_0(t)$	微基站带宽
$g_i^{v2r}(t)$	车辆和路边单元间信道状态
σ^2	平均高斯白噪声方差
$g_j^{r2b}(t)$	路边单元和微基站间信道状态
p_k	路边单元发射功率
f_i^{local}	车辆计算能力
v_i	每 CPU 周期能量消耗系数
$\omega_j(t)$	边缘服务器计算资源分配比例
$\eta_j(t)$	微基站带宽资源分配比例
p_{idle}	车辆空闲时功率
ω_T	时延权重系数
ω_E	能耗权重系数

3.1 系统模型

图 1 构建了 SDN 辅助边云协同的 IoV 系统模型。在该系统模型中,考虑一条双向直线道路,在路边等距放置 M 个路边单元 RSU (Road Side Unit),设置所有 RSU 覆盖区域大小相同,假设每个 RSU 配备 1 台车辆边缘计算服务器 VECS (Vehicle Edge Computing Server),其可以提供存储和计算服务。RSU 通过无线链路与微基站 MBS (Micro Base Station) 相连,进而借助核心网访问云服务器。此外,假设有 N 台车辆在道路上匀速行驶,这些 MV 可以通过 V2I 通信链路向 RSU 传输数据。

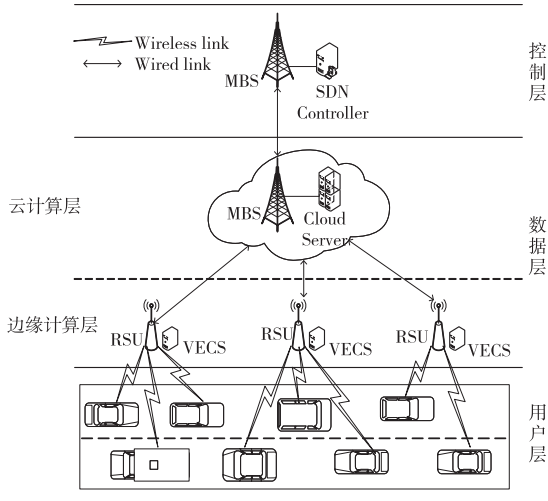


图 1 系统模型

Figure 1 System model

将车辆集合表示为 $\mathcal{N}=\{1,2,\dots,N\}$, RSU 集合表示为 $\mathcal{M}=\{1,2,\dots,M\}$ 。考虑车辆移动性,系统环境是动态变化的,因此,将时间划分为时隙,时隙的集合用 $\mathcal{T}=\{1,2,\dots,T\}$ 表示,其中时隙 $t \in \mathcal{T}$ 长度通常较短(例如 ms)。在时隙 t 内可看作系统环境稳定不变,RSU 覆盖区域内的车辆数量可以被认为是恒定的,表示为 $N_j(t)$ 。 $\mathcal{M}_j$ 指的是第 j 个 RSU,每个 RSU 都连接 1 个 VECS,所以 RSU 和 VECS 的编号是一致的,VECS _{j} 也用来指代与 $\mathcal{M}_j$ 相连的 VECS。在时隙 t 中,第 i 台车辆产生的计算任务可以用 $task_i^t$ 来表示, $task_i^t = (s_i, c_i, t_d)$, s_i 描述任务数据大小(即代码和参数), c_i 表示完成任务所需 CPU 周期总数, t_d 为任务的容忍时延,任务具有足够计算资源且需在容忍时延内完成,否则就会被视为计算失败。

将 SDN 控制器视为协调器,依托 SDN 与网络功能虚拟化技术把 IoV 系统划分成 3 个独立的层,分别为:用户层、数据层以及控制层。所有计算节点由 SDN 控制器统一调度,这些操作均遵循 OpenFlow 协议^[11]。用户层主要由道路上的 MV 组成。数据层则包含了边缘计算节点和云计算节点。控制层由集中式 SDN 控制器组成,包括 2 个子模块,分别为任务监控模块和边云协同模块。任务监控模块负责收集车辆的任务信息(即数据量、计算量和容忍时延等),控制任务在本地计算或卸载到边缘服务器计算亦或是卸载到云服务器计算;边云协同模块负责收集分布式边缘服务器的信息(即服务器有多少内存和 CPU 可用,服务器负载多少)。当车辆生成任务时,会首先向邻近的 RSU 发送任务卸载请求;其次 RSU 会经由 MBS 将请求转发到 SDN 控制器,SDN 控制器综合分析当前全局状况,包括信道状态、可用资源和任务优先级;然后运行计算卸载与资源分配决策方法,得到计算卸载与资源分配决策方案;最后 SDN 控制器会将计算卸载与资源分配决策方案下发到目标计算节点。

3.2 任务卸载模型

任务应该在 1 个时隙内得到处理,否则它将被 VECS _{j} 丢弃^[12],因此不考虑 VECS _{j} 排队时延。假设车辆在时隙 t 内不驶出所属 RSU 覆盖区域,车辆可以在驶出该区域前完成计算任务。车辆的任务卸载决策是相同的,用 $\alpha_i(t)$ 表示^[13], $\alpha_i(t)$ 表示计算负载 s_i 是否在时隙 t 被卸载到 VECS _{j} ,如式(1)所示:

$$\alpha_i(t) = \begin{cases} 0, & \text{计算负载 } s_i \text{ 被卸载到 } VECS_j \\ 1, & \text{计算负载 } s_i \text{ 在本地处理} \end{cases} \quad (1)$$

$VECS_j$ 需要并行处理大量多类型的任务,但 $VECS_j$ 的计算资源是有限的,并且其可用计算资源也是动态变化的。在时隙 t 中, $VECS_j$ 的可用计算资源总量用 $f_j^{\text{free}}(t)$ 来表示。

如果计算负载 s_i 被卸载到 $VECS_j$, 则处理延迟包括从车辆到 RSU(V2R)的传输延迟和 $VECS_j$ 的处理延迟。在任务处理过程中,采用了任务切分策略^[13], $\mathcal{M}_j$ 可以将计算负载 s_i 分为 2 部分,分别由 $VECS_j$ 和云服务器处理。 $VECS_j$ 和云服务器处理 s_i 的切片比例分别为 $1-\beta_i(t)$ 和 $\beta_i(t)$ 。 $\beta_i(t)$ 的取值范围为 $[0,1]$ 。当取 $\beta_i(t)=0$ 时,表示负载 s_i 在 $VECS_j$ 上处理;当取 $\beta_i(t)=1$ 时,表示负载 s_i 在云服务器上处理;当 $\beta_i(t) \in (0,1)$ 时,表示按比例在 $VECS_j$ 和云服务器上处理。

3.3 通信模型

3.3.1 车辆到 RSU

用 $p_i(t)$ 表示第 i 台车辆的传输功率。车辆与 RSU 之间(V2R)无线链路的信道增益用 $g_i^{\text{v2r}}(t)$ 来表示, $g_i^{\text{v2r}}(t) = \xi^{\text{v2r}} d_i^{-r}$, 其中, ξ^{v2r} 是信道小尺度衰落参数, d_i^{-r} 是车辆 i 与 RSU 之间进行数据传输所引起的路径损耗, d_i 是车辆 i 和 RSU 之间的直线距离,而 r 则是路径损耗指数。干扰噪声 $I_i^{\text{v2r}}(t)$ 取决于共享相同信道的其他 V2R 连接,如式(2)所示:

$$I_i^{\text{v2r}}(t) = \sum_{i=1}^{N_j(t)-1} p_i g_i^{\text{v2r}}(t) \quad (2)$$

车辆 i 和 $\mathcal{M}_j$ 之间的平均传输速率如式(3)所示:

$$R_i^{\text{v2r}}(t) = \frac{B_j}{N_j(t)} \log \left[1 + \frac{p_i g_i^{\text{v2r}}(t)}{I_i^{\text{v2r}}(t) + \sigma^2} \right] \quad (3)$$

其中, B_j 是 $\mathcal{M}_j$ 的带宽, σ^2 是平均高斯白噪声的方差。

计算负载 s_i 的平均传输延迟 $T_i^{\text{v2r}}(t)$ 如式(4)所示:

$$T_i^{\text{v2r}}(t) = \frac{[1-\alpha_i(t)]s_i}{R_i^{\text{v2r}}(t)} \quad (4)$$

3.3.2 RSU 到 MBS

考虑 MBS 可用带宽是动态变化的,在时隙 t , 用 $B_0(t)$ 来表示 MBS 的可用带宽。RSU 可以通过 MBS 将计算负载卸载到云服务器,所有 RSU 共享 MBS 可用带宽, $\mathcal{M}_j$ 带宽分配比例为 $\eta_j(t)$ 。 $\mathcal{M}_j$ 和 MBS 之间的信道增益在一个时隙 t 期间保

持恒定,表示为 $g_j^{\text{r2b}}(t)$ 。信号接收功率等于信道增益与传输功率的乘积,所以当存在多个车辆同时卸载计算任务时,车辆 i 通过 $\mathcal{M}_j$ 卸载计算任务到云服务器,其他 RSU 对 $\mathcal{M}_j$ 造成的累计干扰 $I_j^{\text{r2b}}(t)$ 如式(5)所示:

$$I_j^{\text{r2b}}(t) = \sum_{k \neq j, k \in \mathcal{M}} g_j^{\text{r2b}}(t) p_k \quad (5)$$

其中, p_k 是不同 RSU 的发射功率。 $\mathcal{M}_j$ 和 MBS 之间的传输速率 $R_j^{\text{r2b}}(t)$ 如式(6)所示:

$$R_j^{\text{r2b}}(t) = \eta_j(t) B_0(t) \log \left[1 + \frac{p_j g_j^{\text{r2b}}(t)}{I_j^{\text{r2b}}(t) + \sigma^2} \right] \quad (6)$$

如果 $\mathcal{M}_j$ 选择将计算负载 s_i 卸载到 MBS,则从 $\mathcal{M}_j$ 到 MBS 的传输延迟 $T_j^{\text{r2b}}(t)$ 如式(7)所示:

$$T_j^{\text{r2b}}(t) = \frac{[1-\alpha_i(t)]\beta_j(t)s_i}{R_j^{\text{r2b}}(t)} \quad (7)$$

在这里,本文研究了车辆和 RSU 以及 RSU 与云服务器之间通信导致的传输时延。由于下行链路传输时延很小,因此,在本节中忽略了下行链路传输时延。

3.4 计算模型

当车辆产生计算任务时,首先会判断任务所需的计算资源是否超过车辆本地计算能力,若车辆本地可以执行任务,就在本地执行,否则车辆就会将任务卸载到边缘服务器执行。系统根据任务所需计算资源以及 RSU 到 MBS 链路带宽决定是否将任务进一步卸载到云服务器执行。所以,任务执行方式有 3 种,分别为本地执行、边缘执行和云端执行。这 3 种计算方式的定义分别如下文所述。

1) 本地执行。

设定不同的车辆具有相同的计算能力,所有车辆都在本地执行计算任务,用于在车辆 i 本地执行所有计算任务的时延与能耗分别如式(8)和式(9)所示:

$$T_i^{\text{local}} = \frac{c_i}{f_i^{\text{local}}} \quad (8)$$

$$E_i^{\text{local}} = v_i c_i \quad (9)$$

其中, v_i 是每 CPU 周期消耗能量的系数, f_i^{local} 是车辆 i 的计算能力。

2) 边缘执行。

车辆 i 的计算任务通过无线信道卸载到 $\mathcal{M}_j$ 连接的 $VECS_j$ 执行,用于在 $VECS_j$ 上执行车辆 i 计算任务的时延和能耗分别如式(10)和式(11)所示:

$$T_i^{\text{edge}} = T_i^{\text{v2r}}(t) + \frac{c_i}{\omega_j(t) f_j^{\text{free}}(t)} \quad (10)$$

$$E_i^{\text{edge}} = p_i T_i^{\text{v2r}}(t) + p_{\text{idle}} \frac{c_i}{\omega_j(t) f_j^{\text{free}}(t)} \quad (11)$$

其中, p_i 表示车辆的传输功率, $f_j^{\text{free}}(t)$ 表示子时隙 t 的起始时刻 $VECS_j$ 的可用计算资源, $\omega_j(t)$ 表示 $VECS_j$ 给车辆 i 分配的计算资源比例, p_{idle} 表示车辆空闲时的功率。

3) 云端执行。

车辆 i 的计算任务通过无线信道卸载到远程云服务器执行。此情形下, 任务的处理时延由以下部分组成: 计算负载从车辆 i 传输到 $\mathcal{M}_j$ 的时延、计算负载从 $\mathcal{M}_j$ 传输到 MBS 的时延。此外, 由于云服务器强大的计算能力, 本节忽略了任务在云服务器的计算时延。因此, 用于在云服务器执行车辆 i 计算任务的时延和能耗分别如式(12)和式(13)所示:

$$T_i^{\text{cloud}} = T_i^{\text{v2r}}(t) + T_j^{\text{r2b}}(t) \quad (12)$$

$$E_i^{\text{cloud}} = p_v T_i^{\text{v2r}}(t) + p_{\text{idle}} T_j^{\text{r2b}}(t) \quad (13)$$

3.5 任务的优先级模型

为应对不同需求的任务做出迅速决策并找到合适的计算节点, 本节采用为不同类型任务分配优先级的方法。该方法能根据车辆性能及任务数据大小来划分优先级, 实现任务的最佳调度^[14]。通常, 优先级较高的任务更倾向于被卸载到边缘服务器或云服务器上进行计算。任务优先级 pri_i^t 的定义如式(14)所示:

$$\text{pri}_i^t = \frac{g_i^{\text{v2r}}(t)}{s_i \times f_i^{\text{local}} \times p_i} \quad (14)$$

其中, $g_i^{\text{v2r}}(t)$ 表示子时隙 t 内 V2R 链路信道状态, s_i 表示任务数据大小, f_i^{local} 表示车辆计算能力, p_i 表示车辆传输功率。车载任务具有多样性, 车辆可同时产生紧急避险、路径导航和影音娱乐等任务, 这些任务数据大小是不同的, 在同一时刻网络信道状态也是不同的。依据上述优先级公式, 可为不同类型任务划分不同优先级。如紧急避险任务相较其他 2 种类型任务来说数据量较大, 代入式(15)中所得数值最小, 意味着其优先级最高, 需要优先处理。优先调度高优先级任务可确保用户安全, 从而提升用户服务质量。

3.6 系统效用

执行时延的表达式如式(15)所示:

$$TCO_{i,j}^T = \begin{cases} T_i^{\text{local}}, \alpha_i(t) = 1 \\ (1 - \beta_i(t)) T_i^{\text{edge}} + \beta_i(t) T_i^{\text{cloud}}, \\ \alpha_i(t) = 0 \wedge \beta_i(t) \in [0, 1] \end{cases} \quad (15)$$

执行能耗的表达式如式(16)所示:

$$ECO_{i,j}^T = \begin{cases} E_i^{\text{local}}, \alpha_i(t) = 1 \\ (1 - \beta_i(t)) E_i^{\text{edge}} + \beta_i(t) E_i^{\text{cloud}}, \\ \alpha_i(t) = 0 \wedge \beta_i(t) \in [0, 1] \end{cases} \quad (16)$$

其中, $\alpha_i(t)$ 和 $\beta_i(t)$ 分别是车辆 i 在 t 时刻任务的二进制卸载和部分卸载决策。

在前面的小节中, 分别考虑了网络、任务卸载、通信、计算和任务的优先级模型等。在此基础上, 处理车辆 i 计算任务总的时延和能耗如式(17)所示:

$$O_{i,j}^T = \omega_T F(TCO_{i,j}^T) + \omega_E F(ECO_{i,j}^T) \quad (17)$$

其中, ω_T 和 $\omega_E \in [0, 1]$ 分别表示时延和能耗的权重系数。如果 $\omega_E = 0$ 且 $\omega_T = 1$, 则任务是时延敏感的。因此, 为不同类型的任务设定不同的 ω_T 和 ω_E 值。函数 $F(\cdot)$ 起到归一化的作用, 其定义如式(18)所示:

$$F(x) = \frac{x_{\text{local}} - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}} \quad (18)$$

其中, x 用于计算执行时间的输入值和最小值。

IoV 场景中, 制定计算卸载和资源分配决策的主要目标是降低完成任务所需的时延与能耗。但本文提出的基于 DRL 的计算卸载与资源分配决策方法的目标是最大化系统累计奖励。由于优化目标不一致, 故需要定义效用函数来将二者联系起来。综上所述, 定义效用函数如式(19)所示:

$$U_{i,j}^T = -O_{i,j}^T \quad (19)$$

依据上述提及的系统模型与定义, 系统优化目标可以形式化地描述为多目标优化问题 P, 如式(20)所示:

$$\begin{cases} \max: & \sum_{t=1}^T \sum_{i=1}^N \sum_{j=1}^M U_{i,j}^T \\ \text{s.t.} & \text{C1: } \alpha_i(t) \in \{0, 1\}, \beta_j(t) \in [0, 1], \\ & i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T} \\ & \text{C2: } TCO_{i,j}^T \leq t_d, i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T} \\ & \text{C3: } \sum_{i=1}^N \eta_j(t) = 1, i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T} \\ & \text{C4: } \sum_{i=1}^N \omega_j(t) = 1, i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T} \end{cases} \quad (20)$$

其中, C1—C4 是多目标优化问题的 4 个边界条件。具体而言, C1 表示任务的卸载决策; C2 表示完成任务的最晚时间限制; C3 表示带宽分配比例限制; C4 表示计算资源分配比例限制。

命题 1 多目标优化问题 P 的求解是 NP-

hard 问题。

证明 考虑多目标优化问题 P 卸载决策的一种特殊情况,任务的卸载决策只包含 0-1 卸载而不包含部分卸载,则优化目标变为式(21):

$$\begin{cases} \max_{a_i(t), \omega_i(t)} & \sum_{t=1}^T \sum_{i=1}^N \sum_{j=1}^M U_{i,j}^T \\ \text{s. t.} & \text{C1: } \alpha_i(t) \in \{0,1\} \\ & \text{C2: } TCO_{i,j}^T \leq t_d, i \in \mathcal{N}, j \in \mathcal{M}, t \in \mathcal{T} \\ & \text{C3: } \sum_{i=1}^N \omega_i(t) = 1, i \in \mathcal{N}, t \in \mathcal{T} \end{cases} \quad (21)$$

由于卸载决策是二元变量且目标函数是非凸函数,任务计算时延与能耗加权和最小化问题,即系统效用最大化问题,此问题等价于 0-1 背包问题。因此,此特殊情况为 NP-hard 问题,所以,原优化目标 P 也是 NP-hard 问题。□

4 JCORA-DRL 算法

上节所描述的多目标优化问题 P,经过证明是 NP-hard 问题,这是计算复杂度非常高的混合整数非线性规划问题。由于 IoV 中车辆流量的随机性和网络环境的动态性,这个问题用传统的优化方法难以解决。为此,通过 MDP 模型重新表述问题,提出 JCORA-DRL 算法来求解此问题。

JCORA-DRL 算法概览如图 2 所示。位于云端的 SDN 控制器具有收集与分析系统信息和网络瞬态集中控制的功能,位于其上的智能体与系统环境交互做出各种不同决策组合的动作,从而获得环境给的反馈,即奖励。JCORA-DRL 算法中状态空间、动作空间和奖励函数的设计分别如下所描述。

状态空间: t 时刻,车辆 i 产生的计算任务 $task_i^t$ 、任务优先级 pri_i^t 、RSU 和 MBS 之间的信道增益 $g_j^{r2b}(t)$ 、VECS _{j} 可用计算资源 $f_j^{\text{free}}(t)$ 和 MBS 可用带宽 $B_0(t)$ 。 t 时刻状态空间定义为式(22):

$$s_t = \{task_i^t, pri_i^t, f_j^{\text{free}}(t), g_j^{r2b}(t), B_0(t)\} \quad (22)$$

动作空间:位于 SDN 控制器的智能体与系统环境进行交互做出计算卸载与资源分配决策组合动作。其决策组合包括二进制卸载决策 $\alpha_i(t)$ 、部分卸载决策 $\beta_j(t)$ 、MBS 带宽分配比例 $\eta_j(t)$ 以及 VECS _{j} 计算资源分配比例 $\omega_j(t)$ 。 t 时刻的动作空间可描述为式(23):

$$a_t = \{\alpha_i(t), \beta_j(t), \eta_j(t), \omega_j(t)\} \quad (23)$$

奖励函数:智能体在状态 s_t 下,执行动作 a_t 后

获得奖励 r_t 。训练能否实现收敛主要受奖励函数的影响。车辆边缘计算系统设计中,主要目标是在满足多目标优化问题 P 所设立的约束条件的同时,尽量减少系统在时延与能耗方面的开销。因此,将时刻 t 系统的即时奖励 r_t 定义为式(24):

$$r_t = \begin{cases} U_{i,j}^T * \frac{1}{pri_i^t}, & \text{任务计算成功} \\ -CO_{i,j}^T, & \text{任务计算失败(不满足容忍时延)} \end{cases} \quad (24)$$

其中, $-CO_{i,j}^T$ 是任务计算失败的成本。

在时刻 t ,智能体观察系统环境状态 s_t ,依据奖励 r_t ,通过 JCORA-DRL 算法得到计算卸载与资源分配决策最优动作 a_t 。

JCORA-DRL 算法中使用 DDPG(Deep Deterministic Policy Gradient)算法用于训练深度神经网络 DNN(Deep Neural Network),以获得最佳的计算卸载与资源分配决策。DDPG 算法使用独立的网络结构,这种设计能够显著降低数据间的相互依赖性,从而提高算法的稳定性,并进一步增强模型的鲁棒性。智能体和系统环境进行交互获得的训练数据样本具有较强的相关性,不是独立同分布的,所以在算法中引入经验重放机制,这可以有效降低训练数据的相关性。

DDPG 算法包含 4 个 DNN,它们分别是 Actor_主网络 $\mu(s | \theta^\mu)$, Critic_主网络 $Q(s, a | \theta^Q)$, Actor_目标网络 $\mu'(s | \theta^{\mu'})$, Critic_目标网络 $Q'(s, a | \theta^{Q'})$,其中 $\theta^\mu, \theta^Q, \theta^{\mu'}, \theta^{Q'}$ 分别表示 4 个网络的可学习参数。Actor_主网络的任务是依据系统当前状态挑选要执行的计算卸载和资源分配决策组合动作,而 Critic_主网络的任务则是对 Actor_主网络所选出的动作进行评估,计算其对应的 Q 值。因为评估的 Q 值可以用来指导 Actor_主网络学习最佳的决策,所以使用 Critic_主网络能够减少策略梯度的误差。Critic_目标网络和 Actor_目标网络使用经验缓冲器中的数据来评估目标 Q 值。Critic_主网络负责计算 Q 值, t 时刻目标 Q 值 Q^{target} 的表达式如式(25)所示:

$$Q_t^{\text{target}} = r_t + \gamma Q'(s_{t+1}, \mu'(s_{t+1} | \theta^{\mu'}) | \theta^{Q'}) \quad (25)$$

其中, γ 表示奖励折现因子。

Critic_主网络的参数更新,需要使用梯度下降算法来最小化 Q_t^{target} 与 Critic_主网络输出之间的差异。损失函数采用平方损失函数,如式(26)所示:

$$L_s = \frac{1}{P} \sum_{i=1}^P (Q_i^{\text{target}} - Q(s_i, a_i | \theta^Q))^2 \quad (26)$$

算法 1 JCORA-DRL 算法

输入: 车辆计算能力 f_i^{local} , VECS_j 计算资源 f_j^{free} , MBS 带宽 $B_0(t)$, 采样频率 f_{sample} , 批处理样本大小 G 。

输出: 卸载决策 $\alpha_i(t)$ 和 $\beta_i(t)$, 带宽分配比例 $\eta_j(t)$ 和计算资源分配比例 $\omega_j(t)$ 。

```

1. 随机初始化 actor_主网络参数  $\theta^a$  和 critic_主网络参数  $\theta^c$ , 初始化 actor_目标网络参数  $\theta^{a'} \leftarrow \theta^a$  和 critic_目标网络参数  $\theta^{c'} \leftarrow \theta^c$ ;
2. 初始化经验回放池  $R$ ;
3. for  $episode = 1, 2, \dots, E$  do
4.   获取环境的初始状态  $s_1$ ;
5.   for  $t = 1, 2, \dots, T$  do
6.     根据 actor_主网络和噪声, 获取动作  $a_t$ ;
7.     if  $\alpha_i(t) = 0$  then
8.       if  $\beta_i(t) = 0$  then
9.          $r_t = \text{vec. work\_edge}(\alpha_i(t), \beta_i(t), \omega_i(t))$ ;
          //任务在 VECSj 完成计算, 得到即时的奖励
10.      else if  $\beta_i(t) = 1$  then
11.         $r_t = \text{vec. work\_cloud}(\alpha_i(t), \beta_i(t), \eta_i(t))$ ;
          /* 任务在云服务器完成计算, 得到即时的奖励 */
12.      else
13.         $r_{t1} = \text{vec. work\_edge}(\alpha_i(t), \beta_i(t), \omega_i(t))$ ;
14.         $r_{t2} = \text{vec. work\_cloud}(\alpha_i(t), \beta_i(t), \eta_i(t))$ ;
15.         $r_t = r_{t1} + r_{t2}$ ; /* 任务在 VECSj 和云服务器上完成计算, 得到即时奖励 */
16.      else  $\alpha_i(t) = 1$  then
17.         $r_t = \text{vec. work\_local}(\alpha_i(t))$ ; /* 本地执行, 得到即时的奖励 */
18.      执行动作  $a_t$  后, 得到奖励  $r_t$  和下一个状态  $s_{t+1}$ ;
19.      将  $(s_t, a_t, r_t, s_{t+1})$  转换成经验存入经验缓冲区  $R$  中;
20.      if  $t > W$  and  $t \bmod f_{\text{sample}} = 0$  then
21.        for  $\tilde{g} = 1, 2, \dots, G$  do
22.          根据绝对时序差分误差计算经验  $j$  的优先级;
23.          计算经验  $j$  的采样概率  $P(j)$ ;
24.          计算相应的重要性采样权重;
25.        end for
26.      从  $R$  中随机抽取  $B$  个样本;
27.      用式(25)计算目标  $Q$  值;
28.      基于式(26)通过最小化损失函数来更新 critic_主网络参数;
29.      使用基于式(27)的采样策略梯度更新 actor_主网络策略;
30.      基于式(28)和式(29)对目标网络参数进行软更新;
31.    end for
32.  end for
33. return  $\alpha_i(t), \beta_i(t), \eta_j(t), \omega_j(t)$ ;

```

考虑 4 个 RSU, 每个 RSU 覆盖 3 台车辆, RSU 的半径为 250 m。参考文献[13, 19, 20]中的实验参数设置, 本文仿真实验参数设置如表 2 所示。

表 2 IoV 参数设置

参数名	参数值
RSU 数量	4
$task_i^t$ 数据大小/MiB	[5, 10]
$task_i^t$ 计算量/GHz	[2, 15]
$task_i^t$ 容忍时延/ms	[5, 40]
车辆计算能力/GHz	5
VECS 计算资源/GHz	25
车辆传输功率/W	0.2
平均高斯白噪声/dBm	-100
RSU 带宽/MHz	20
RSU 传输功率/W	2
MBS 带宽/MHz	40
处理 1 位数据需要的 CPU 周期数	300
时延权重系数	0.5
能耗权重系数	0.5

本文的实验是在配备 8 核 Intel® Core™ i7-7700H CPU 的电脑上进行的, 使用 24 GiB 内存的 NVIDIA® GeForce RTX™ 3090 GPU 和 CUDA 12.1 来训练 DRL 模型, 此方法运行在 Python 3.8 和 PyTorch 1.7 上。

DDPG 算法由 4 个神经网络组成, 每个神经网络包括 2 个隐藏层, 每个隐藏层具有 256 个神经元, 在每个隐藏层上还添加了非线性激活函数。模型训练了 5 000 个训练回合 (episode), 每个 episode 包括 100 个步骤。软更新参数 τ 和奖励折现因子 γ 分别设置为 0.01 和 0.95。DRL 模型优化器为 Adam。演员和评论家网络的学习率分别设置为 0.001 和 0.002。经验缓冲器大小设置为 100 000。批处理大小 ($batch_size$) 设置为 64。

5.2 算法训练过程

首先, 对演员和评论家网络的参数进行初始化, 通过演员网络产生动作, 并在环境中执行这些动作以获得奖励和下一个状态, 将执行过程中的 (状态、动作、奖励和下一个状态) 存储到记忆库中; 其次, 从记忆库中随机采样一批经验数据, 使用这批数据更新评论家网络, 通过评论家网络计算目标 Q 值, 并计算损失函数来更新评论家网络, 使用评论家网络来计算当前策略的梯度, 根据计算出的策略梯度更新演员网络; 然后, 以一定的时间间隔, 采用

软更新的方式,使用主网络的参数来更新目标网络的参数;最后,判断是否达到预设的训练时间或性能指标,若没有达到,则返回继续,若达到目标,当训练完成或达到终止条件时,结束并输出训练结果。

5.3 对比实验

为了评估本文所提出方法的性能,选取以下 4 种基准算法和 2 种较新算法,来检验本文提出的 JCORA-DRL 算法的有效性。

1) Local 算法:车辆的计算任务都在车辆本地进行处理。

2) Edge 算法:车辆的计算任务均卸载到 VECS 进行处理。

3) Cloud 算法:车辆的计算任务均通过 RSU 卸载到云服务器进行处理。

4) Random 算法:车辆的计算任务卸载决策随机在车辆、VECS 和云服务器中选择,计算资源和带宽资源分配比例也是随机选择的。

5) SAC(Soft Actor-Critic)算法^[21]:车辆的计算任务通过使用 SAC 算法来产生计算卸载与资源分配决策。

6) TD3(Twin Delayed Deep Deterministic policy gradient)算法^[13]:车辆的计算任务采用 TD3 算法来生成计算卸载与资源分配决策。

首先,本节研究了算法参数对训练过程奖励的影响。如图 3a 所示,批处理大小为 64 的算法获得的奖励最高,在下文中,算法的训练过程都选择批处理大小为 64。同样,如图 3b 所示,奖励折扣因子 γ 为 0.95 的算法奖励效果最好。此外,从图 3c 中可以看出,不同学习率对算法训练过程奖励的影响较小。由此可得,本节的算法对学习率的设置并不敏感。综上,选择批处理大小为 64 和奖励折扣因子 γ 为 0.95 进行算法的训练。同时,Adam 优化器在演员网络学习率 lr_{actor} 为 0.001 和评论家网络学习率 lr_{critic} 为 0.002 的情况下更新网络参数。

处理不同车辆数量下的计算任务并经过 5 000 次迭代计算后,4 种基准算法和 2 种较新算法的计算时延如图 4a 所示。JCORA-DRL 算法相较于 4 种基准算法和 2 种较新算法而言,其时延分别降低了 91.88%、76.74%、85.44%、59.15%、39.2%和 13.49%。

图 4b 展示了各处理方式的系统能耗随车辆数量的变化趋势。由图可知,系统能耗也和时延一样呈现类似增长特征。JCORA-DRL 算法系统能耗相较于 4 种基准算法和 2 种新近算法分别降低了 23.38%、18.35%、55.29%、8.52%、18.01%和 6.56%。

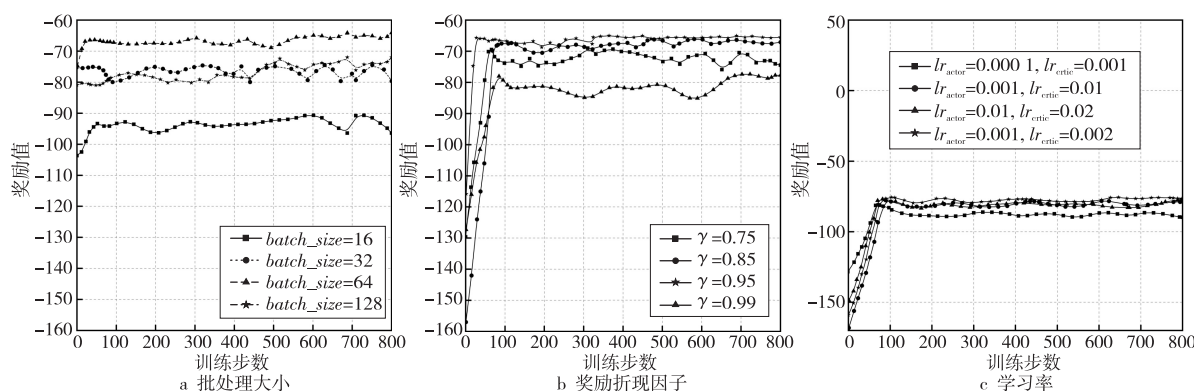


图 3 不同算法超参数下的奖励:批处理大小、奖励折现因子和学习率

Figure 3 Reward under different algorithm hyper parameters: batch size, reward discount factor, and learning rate

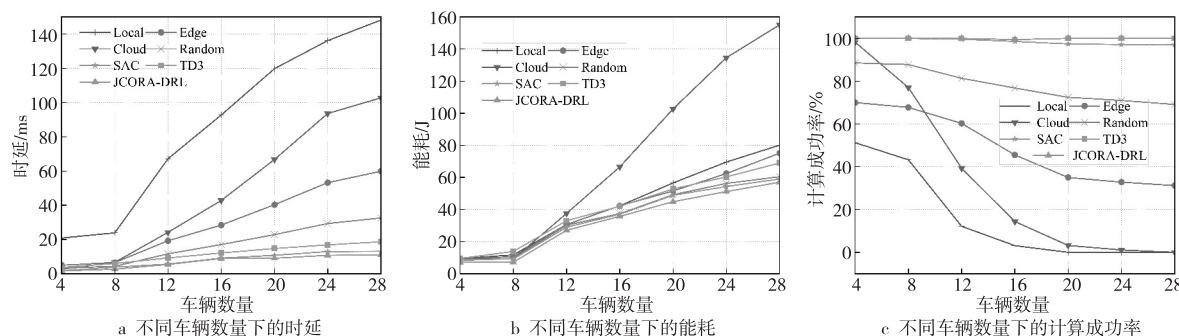


图 4 不同车辆数量下的时延、能耗和计算成功率

Figure 4 Time delay, energy consumption, and calculation success rate for different number of vehicles

图 4c 展示了各处理方式的任务计算成功率随车辆数量的变化趋势。Local 算法平均任务计算成功率为 15.69%。Cloud 算法在小于等于 8 台车辆的时候,其平均计算成功率为 87.63%。Random 和 Edge 这 2 种算法的平均计算成功率分别为 78.13%和 48.93%。JCORA-DRL 算法平均计算成功率始终维持在 99.92%,SAC 算法平均计算成功率为 98.53%,而 TD3 算法的平均计算成功率为 95.97%。这是因为 JCORA-DRL 算法使用确定性策略,在给定状态下,它总是选择相同的最佳动作。这种确定性不仅可以减少由于随机性引入的误差,而且可以使得算法更容易收敛到稳定状态,从而维持较高任务计算成功率。

图 5a 展示了各处理方式的系统时延随着 MBS 网络带宽的变化趋势。Local 算法的计算时延一直为 66.58 ms。Edge 和 Random 算法,平均值分别为 20.31 ms 和 11.21 ms。Cloud 算法计算时延平均值为 26.96 ms。JCORA-DRL 算法和 2 种强化学习算法的计算时延平均值分别为 5.39 ms,5.79 ms 和 9.36 ms。JCORA-DRL 算法的时延相较于 4 种基准算法和 2 种新近算法的分别降低了 91.91%、73.47%、80.01%、51.92%、42.43% 和 6.96%。

图 5b 展示了各处理方式的系统能耗随着 MBS

网络带宽的变化趋势。JCORA-DRL 算法能耗相较于 4 种基准算法和 2 种新近算法分别降低了 35.11%、12.9%、20.42%、-5.88%、0.78% 和 7.6%。

图 5c 展示了各处理方式的计算成功率随着 MBS 网络带宽的变化趋势。本文将任务计算成功率定义为处理时延不超过容忍时延的任务数与所有任务数之比。Local 算法的平均计算成功率为 2.96%,Cloud 算法的平均计算成功率为 14.11%,Edge 和 Random 算法和平均计算成功率分别为 55.75%和 82.6%。

SAC、JCORA-DRL 和 TD3 的平均计算成功率分别为 98.86%、96.25%和 94.01%。SAC 算法通过熵正则化促进策略探索,随着 MBS 带宽增加,这种探索会帮助算法发现更多有效的任务卸载决策,也会使得 SAC 算法更专注于控制任务卸载以达到长期较高的计算成功率。JCORA-DRL 算法使用的确定性策略减少由随机性引起的额外时延,这在带宽增加时有助于快速决策,提高计算成功率。TD3 算法目标网络更新频率低,这会造成目标网络策略落后于主网络,这种延迟会影响任务卸载到其他计算节点执行时的性能,从而影响计算成功率。

图 6a 展示了 ES 计算能力对不同方法系统时延的影响。JCORA-DRL 算法相较于 4 种基准算法和 2 种较新算法,时延分别降低了 91.95%、69.61%、

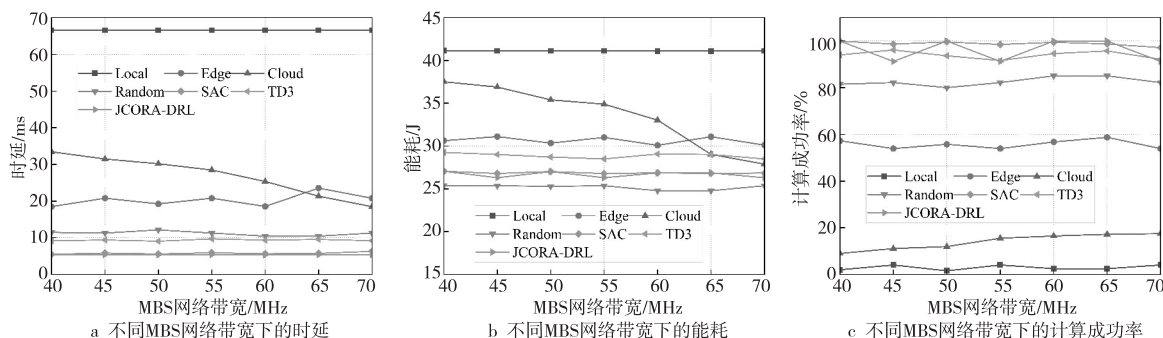


图 5 不同 MBS 网络带宽下的时延、能耗和计算成功率

Figure 5 Time delay, energy consumption, and calculation success rate for different MBS network bandwidths

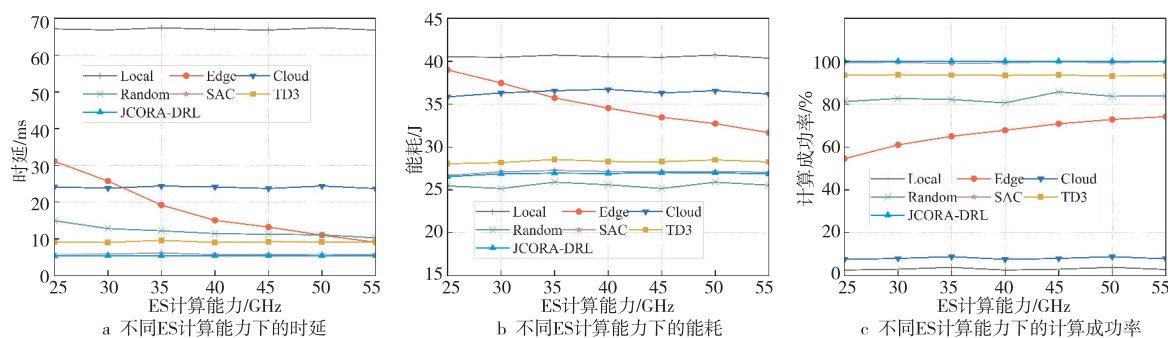


图 6 不同 ES 计算能力下的时延、能耗和计算成功率

Figure 6 Time delay, energy consumption, and calculation success rate for different ES computing capabilities

77.54%、55.01%、2.66%和 41.31%。

图 6b 体现了不同 ES 计算能力下系统能耗的变化。JCORA-DRL 算法相较于 4 种基准算法和 2 种新近算法,能耗分别降低 33.71%、23.08%、26.07%、-5.28%、0.41%和 5.03%。

图 6c 给出了不同 ES 计算能力下的任务计算成功率。结果表明,Edge 算法平均计算成功率可达到 66.61%。Local 算法平均计算成功率为 3.1%。Cloud 算法,平均计算成功率为 8.1%。Random 算法平均计算成功率为 82.85%。JCORA-DRL 算法和 2 种新近算法的平均计算成功率分别为 100%, 99.62%和 93.46%。因为 JCORA-DRL 算法使用确定性策略,这可以为任务卸载提供明确的动作决策,减少由于策略不确定性带来的失败风险,有助于维持较高任务计算成功率。

图 7 分别展示了不同训练步数对奖励值和训练损失的影响。由图可知,JCORA-DRL 算法奖励值随着训练步数增加逐渐提高,最终趋于稳定。训练损失会随着训练步数的增加而逐渐降低,在有限的训练步数内能够达到稳定收敛。JCORA-DRL 算法相较于 4 种基准算法和 2 种新近算法,奖励值分别提高了 83.46%、61.8%、80.04%、89.06%、78.74%和 78.9%。从奖励值可以直观看出本文使用的 JCORA-DRL 算法在解决计算卸载与资源分配问题上具有优势,这有效提升了系统的性能。

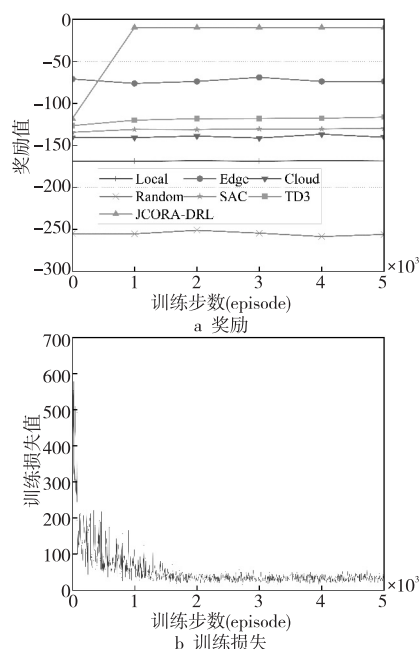


图 7 不同训练步数下的奖励和训练损失
Figure 7 Rewards and training losses for different training steps

6 结束语

本文研究了在 IoV 系统中边云协同下的计算卸载与资源分配问题。首先给出了协同计算卸载的网络架构。其次构建了包括网络模型、通信模型、任务卸载模型、计算模型、任务优先级模型和系统效用模型在内的系统模型体系。然后通过联合优化计算任务的时延与能耗以及考虑任务的计算成功率,提出了一种基于 DDPG 算法的决策方法。最后通过仿真实验验证了 JCORA-DRL 算法比 4 种基准算法和 2 种较新算法性能更优,在降低任务处理时延与能耗以及提高任务计算成功率方面有更好的性能。此外,由于 DDPG 算法的探索机制不够高效,这可能导致其在训练期间需要大量的样本来学习有效的策略。在车联网中,车辆不仅是单一的智能体,还需要与周围其他智能体进行协调。DDPG 算法作为一种单智能体学习方法,可能需要进一步的扩展才能有效处理多智能体之间的相互作用。未来的工作,本文将考虑在分布式车联网系统场景下,采用多智能体强化学习算法来解决计算卸载与资源分配问题。

参考文献:

- [1] REN Y L, CHEN X Y, GUO S, et al. Blockchain-based VEC network trust management: A DRL algorithm for vehicular service offloading and migration [J]. IEEE Transactions on Vehicular Technology, 2021, 70(8): 8148-8160.
- [2] 陈琳, 董甲东, 潘凯, 等. 云边端和 D2D 边缘架构下的计算卸载策略综述[J]. 计算机工程与应用, 2023, 59(15): 55-67.
CHEN Lin, DONG Jiadong, PAN Kai, et al. Survey of computation offloading strategies under cloud-edge-end and D2D edge architectures [J]. Computer Engineering and Applications, 2023, 59(15): 55-67.
- [3] LU L Y, YU J, DU H F, et al. A3C-based load-balancing solution for computation offloading in SDN-enabled vehicular edge computing networks[J]. Peer-to-Peer Networking and Applications, 2023, 16(2): 1242-1256.
- [4] ZHANG Z, ZENG F. Efficient task allocation for computation offloading in vehicular edge computing [J]. IEEE Internet of Things Journal, 2023, 10(6): 5595-5606.
- [5] 刘继军, 邹山花, 卢先领. MEC 中资源分配与卸载决策联合优化策略[J]. 计算机科学与探索, 2021, 15(5): 848-858.

- LIU Jijun, ZOU Shanhua, LU Xianling. Joint optimization scheme of resource allocation and offloading decision in mobile edge computing[J]. Journal of Frontiers of Computer Science and Technology, 2021, 15(5):848-858.
- [6] ZHAO J H, LI Q P, GONG Y, et al. Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks[J]. IEEE Transactions on Vehicular Technology, 2019, 68(8): 7944-7956.
- [7] 杜梅, 周军华, 李敦桥, 等. MEC 计算卸载与资源分配联合智能优化方案[J]. 北京邮电大学学报, 2022, 45(2):65-71.
- DU Mei, ZHOU Junhua, LI Dunqiao, et al. A joint intelligent optimization scheme of computation offloading and resource allocation for MEC[J]. Journal of Beijing University of Posts and Telecommunications, 2022, 45(2):65-71.
- [8] CHEN C, LI H F, LI H, et al. Efficiency and fairness oriented dynamic task offloading in Internet of Vehicles[J]. IEEE Transactions on Green Communications and Networking, 2022, 6(3):1481-1493.
- [9] ZHANG H B, WANG Z X, LIU K J. V2X offloading and resource allocation in SDN-assisted MEC-based vehicular networks[J]. China Communications, 2020, 17(5):266-283.
- [10] 王姗姗, 罗志勇. 基于云边协同算力调度的高效边缘卸载研究[J]. 无线电通信技术, 2023, 49(1):56-62.
- WANG Shanshan, LUO Zhiyong. Research on efficient edge offloading based on cloud-edge collaboration computing power scheduling [J]. Radio Communications Technology, 2023, 49(1):56-62.
- [11] YANG L, NG B, SEAH W K G, et al. A survey on network forwarding in software-defined networking [J]. Journal of Network and Computer Applications, 2021, 176: Article No. :102947.
- [12] YE Q, SHI W S, QU K G, et al. Joint RAN slicing and computation offloading for autonomous vehicular networks: A learning-assisted hierarchical approach[J]. IEEE Open Journal of Vehicular Technology, 2021, 2:272-288.
- [13] HUANG J W, WAN J Y, LV B F, et al. Joint computation offloading and resource allocation for edge-cloud collaboration in Internet of Vehicles via deep reinforcement learning[J]. IEEE Systems Journal, 2023, 17(2):2500-2511.
- [14] 熊兵, 张俊杰, 黄思进, 等. 多约束边环境下计算卸载与资源分配联合优化[J]. 小型微型计算机系统, 2024, 45(2):405-412.
- XIONG Bing, ZHANG Junjie, HUANG Sijin, et al. Joint optimization of computational offloading and resource allocation in multi-constraint edge computing[J]. Journal of Chinese Computer Systems, 2024, 45(2):405-412.
- [15] LI C, GROSSMANN I E. A review of stochastic programming methods for optimization of process systems under uncertainty[J]. Frontiers in Chemical Engineering, 2021, 2: Article ID:622241.
- [16] TAN H N. Reinforcement learning with deep deterministic policy gradient[C]//Proceedings of the 2021 International Conference on Artificial Intelligence, Big Data and Algorithms, 2021:82-85.
- [17] LILLICRAP T P, HUNT J J, PRITZEL A, et al. Continuous control with deep reinforcement learning [J]. arXiv:1509.02971, 2015.
- [18] SEHLA K, NGUYEN T M T, PUJOLLE G, et al. Resource allocation modes in C-V2X: From LTE-V2X to 5G-V2X[J]. IEEE Internet of Things Journal, 2022, 9(11):8291-8314.
- [19] HAZARIKA B, SINGH K, BISWAS S, et al. DRL-based resource allocation for computation offloading in IoV networks[J]. IEEE Transactions on Industrial Informatics, 2022, 18(11):8027-8038.
- [20] WANG S M, SONG X Q, XU H, et al. Joint offloading decision and resource allocation in vehicular edge computing networks [J]. Digital Communications and Networks, 2023, 11(1):71-82.
- [21] ZHENG Y L, ZHOU H, CHEN R, et al. SAC-based computation offloading and resource allocation in vehicular edge computing[C]//Proceedings of the 2022 IEEE Conference on Computer Communications Workshops, 2022:1-6.

作者简介:



张文柱(1970—),男,辽宁葫芦岛人,博士,教授,研究方向为无线通信理论与技术和移动边缘计算。E-mail: wzzhang@xauat.edu.cn

ZHANG Wenzhu, born in 1970, PhD, professor, his research interests include wireless communication theory & technology and mobile edge computing.



石亚坤(1997—),男,陕西安康人,硕士生,研究方向为移动边缘计算。E-mail: shiyakun@xauat.edu.cn

SHI Yakun, born in 1997, MS candidate, his research interest includes mobile edge computing.